

# SI 630 Homework 4: Prompt-based NLP

## 1 Introduction

Training an NLP model can require lots of training data! But what if we didn't need a lot of labeled data? More recent pre-trained language models have focused on instruction fine-tuning [1] where the text that the language model is trained on contains many examples of instructions to follow and the desired output of those instructions. When models are pre-trained on lots of text and are fine-tuned to follow instructions, this lets them perform a variety of tasks on a surprising variety of topics (though this is dependent on what kinds of instructions they're trained to follow!).

The IFT prompt-based models allow you, as a practitioner, to do zero-shot learning where a model is only given a prompt to follow and is asked to generate text that can be turned into an answer. If it works, zero-shot learning offers huge advantages since we no longer need to create massive datasets and can instead get by with a potentially much smaller set of examples. The central questions are then how to craft prompts.

In Homework 4, you will explore prompting with a few different open source models and test on a small set of items from one dataset, HellaSwag [2], which is a multiple-choice sentence completion task that requires common sense reasoning. Here, an example context is shown and four possible endings are given; for example, given the context:

Then, the man writes over the snow covering the window of a car, and a woman wearing winter clothes smiles. then

a model must pick from:

1. the man adds wax to the windshield and cuts it.
2. a person board a ski lift, while two men supporting the head of the person wearing winter clothes snow as the we girls sled.
3. the man puts on a christmas coat, knitted with netting.
4. the man continues removing the snow on his car.

HellaSwag was designed and curated specifically to be adversarial to models—but easy for humans.

When using zero-shot prompting your task is to design a prompt that elicits an answer that can be mapped to some label. For example, with HellaSwag, you might give the model

four numbered options and have it print which specific number could finish the sentence. Or, you could make the list be with letters! Or have it just repeat the text of the right answer! Or you could ask it to give you a number and also explain why after! Or you could have it generate JSON output that you parse to get the answer! All of these kinds of designs are possible, including how you phrase the question. This makes for a huge design space in terms of how you elicit answers. Homework 4 has you explore this design space to see how your choices impact model performance.

This assignment has the following learning goals:

- Familiarize you with the idea of zero-shot learning
- Learn how to generate text using huggingface
- Learn how to perform zero-shot classification

This last assignment is aimed at giving you one more skill in your arsenal for when you need an NLP classifier but don't have labeled data and can't create a large dataset of labeled examples.

## 2 Designing Prompts

Prompt design is more of an art than a science. A simple prompt for sentiment analysis might be:

“Consider the following sentence: [text]: Does the sentence have a positive or negative sentiment? Answer positive or negative.”

Yet prompts can become much more complex, include instructions and examples, and have explicit structure (like Markdown or LaTeX) for the model to interpret. There are many good guides for how to create effective prompts for NLP.<sup>1</sup>

For example, Ziems et al. [2024] highlight a few guidelines collected from other papers:

- When the answer is categorical, enumerate options as alphabetical multiple-choice so that the output is simply the highest-probability token ('A', 'B').
- Each option should be separated by a new line to resemble the natural format of online multiple-choice questions.
- To promote instruction-following, give instructions after the context is provided; then explicitly state any constraints.
- Clarify the expected output in the case of uncertainty.

---

<sup>1</sup><https://huggingface.co/docs/transformers/main/en/tasks/prompting>

<https://zapier.com/blog/gpt-prompt/>

<https://www.statworx.com/en/content-hub/blog/paradigm-shift-in-nlp-5-approaches-to-write-better-prompts/>

<https://www.reddit.com/r/PromptDesign/>

- When the answer should contain multiple pieces of information, request responses in JSON format.

These are certainly non-exhaustive and may not all apply well to the current task. Your job as a practitioner will be to explore the design space for your particular problem.

Because prompt design is just writing text, the practitioner has to think about both what skills a model might already know and what types of instructions have been explicitly spelled out in the design. Often for complex tasks, the model may benefit from including detailed instructions or descriptions, e.g., nuanced definitions of each class in a multiclass classification. Yet, depending on the model, longer prompts may cause the model to go off track in what the task is!

**Chain of Thought (CoT) Prompts** One of the early observations in prompting is to include instructions for a model to “think step by step” or “let’s think carefully” [3]. Perhaps due to data in pre-training, asking a model to generate answers carefully was shown to produce more accurate results. Asking a model to generate step-by-step instructions as well can show its rationale and often generate better answers.

**In Context Learning Prompts** Although we aren’t training a model, one powerful way of using training data is with in-context learning. Here, the prompt instructions include examples of input and correct outputs so that the model can see how to perform the task. Usually, such prompts include a few examples for the task.

## 3 Data

We’ve given you three files, but you only need two of them:

- `hellaswag_val.300.jsonl` — Use your prompts to generate predictions for the 300 instances in this file and then use the labels in the file to score your different prompt+model performances.
- `hellaswag_test2.1k.student.json` — This is the file that contains unlabeled instances you’ll generate predictions for and then upload those to Kaggle InClass.
- `hellaswag_train.jsonl` — In case you want more data, feel free to get examples to train/explore from this file. You do not need to use it for any graded parts of the assignment.

## 4 Your Tasks

Homework 4 has three high-level tasks. First, you’ll be writing a few different kinds of instructions to go in the model prompts in order to classify the right sentence ending for HellaSwag. The models (listed below) will each have specific formatting requirements so you’ll need to see how to retrofit your instructions into the model prompt. The second task

is to write the code that prompts the model to answer a question and then save the answer-predictions as output. You'll score these outputs and make a nice plot. Third, you'll write a reflection on your prompt design and effort.

For all of the prompt designs, you are allowed and encouraged to get creative. Feel free to read online guides on effective prompting (there are many). You'll want to think about length too—long prompts are slower and with the input may be longer than a context window! But maybe you need more details; it's always a trade-off and some of the models we've included do have longer context windows. You are not evaluated on how well your prompts do; we're specifically encouraging you to try things out here.

**Problem 1. (10 points; code)** After extensive prototyping, write and choose your three best prompt instructions that you think should work well. The three instructions should be sufficiently different in text. At least one of your prompts should use some kind of Chain of Thought instructions.

**Problem 2. (5 points; code)** How important are the instructions? To test this, write a minimal prompt that still generates a valid answer for an input but includes almost no instructions. Your prompt should still have the input sentence in it at least.

**Problem 3. (5 points; code)** Design one in-context learning prompt that has at least two examples in it. Do your best to fit everything in the maximum context window size.

**Problem 4. (10 points; PDF)** Write a paragraph-length reflection on how you approached designing your and what kind of prototyping you did. What makes you think these prompts might work or not work? For the good prompts, what kinds of approaches did you try that didn't work? If you read any guides, include these here and indicate which rules or guidance you found helpful.

We'll be trying out six different models, one of which is going to be a bad non-IFT baseline: GPT2. The others have some form of IFT. To run each of these, look at the Huggingface model page and get the code from the page. Each of the models is slightly different in what it requires. For example, the Flan-T5 model is an encoder-decoder and so requires a different AutoModel class, while the Mistral model needs a very specific kind of prompt format to work well. Your job will be to format your instructions into the right prompt format for each.

1. GPT2 <https://huggingface.co/openai-community/gpt2>
2. Gemma-2b <https://huggingface.co/google/gemma-2b>
3. flan-t5-large <https://huggingface.co/google/flan-t5-large>
4. OPT-2.7b <https://huggingface.co/facebook/opt-2.7b>
5. Mistral-7B-Instruct <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.1>

2

We strongly encourage trying these models locally on your CPU with a sentence or two. They'll run just fine, though they may be slow. If the code runs on CPU, then it should run on a GPU when you deploy it to great lakes. Some of these models also may have smaller versions that can run on a CPU faster; you might try debugging first with `flan-t5-small` or `GPT2`, which can generate output (though it might not be any good!). Some models also support working in mixed-precision, i.e., instead of using 32 bit floats, they can run with 16 or even 8 bit floats. You are welcome to use those versions of the models—they'll run faster—so please see on the model page if there are instructions. However, the models will still work just fine if you don't use mixed-precision.

**Problem 5. (15 points; code/PDF)** Evaluate your five prompts with the six models. Since the dataset is balanced across class labels (roughly), score models using accuracy. Make one plot for each model that shows each of the five prompt's accuracy scores as colored bars (use the same colors across plots). In a few sentences describe what trends you see in the plot.

**Problem 6. (2.5 points; code; PDF)** For your best-performing prompt and model, generate the predictions for the held-out test set. In one sentence describe in the PDF how you produced these results (what prompt and model).

**Problem 7. (12.5 points; PDF)** Having tested models and prompts, write a two paragraph reflection on prompt design, what you learned from this exercise, and how you would apply what you learned if asked to generate prompts for other types of tasks (including those that aren't multiple choice!). Include a list of at least three specific pieces of advice you would give a colleague on how to design prompts.

## 5 Options Parts (may speed things up)

Disclaimer: You don't need to do any of these pieces. However, when doing zero-shot prompting, speed is of the essence since you, as the practitioner, likely have to include significant amounts of instructions or examples and the Transformer is  $O(n^2)$  in sequence length.

- Run your decoder models using `vLLM` <https://github.com/vllm-project/vllm>, which is an alternative library to HuggingFace that can still reuse the model parameters. Note that this won't work for encoder-decoder models like `flan-t5`. You will likely see significant speed up in practice—though you have to get it running first.
- If you're running on a machine that supports it, you can try to use the quantized CPU-versions of the models with `Ollama` <https://ollama.com/>. These may not be as fast as a GPU, but you won't need a GPU so you could potentially run them overnight. Not all models are supported though!

- You can also try forcing the model to load in quantized or lower-precision versions directly. Some of this may require packages like `bitsandbytes`. See this guide for example on how to run Mistral locally on your laptop: <https://cheatsheet.md/llm-leaderboard/how-to-run-mistral-locally>. The instructions should still work with a GPU and it will likely be faster.

## 6 Great Lakes

This homework requires that you use a GPU for generating the answers with the bigger models. You can use your own or you can use the GPUs on Great Lakes for free. We have provided you with a course account for Great Lakes, the same one from Homework 3, which will give you access for many hours. Your Great Lakes jobs are limited to 4 hours of training, which is more than enough to generate answers for this assignment.

In case it's still helpful, here is the guide from Homework 3 on how to access Great Lakes, set up your environment, and submit your jobs: Great Lakes Tutorial.

## 7 What to Submit

You should submit the following parts to Canvas:

- Your code for all parts in a `.ipynb` file with outputs visible
- A corresponding `.html` file
- A pdf containing your answers to explanation questions.

## 8 Academic Honesty

Unless otherwise specified in an assignment, all submitted work must be your own, original work. Any excerpts, statements, or phrases from the work of others must be clearly identified as a quotation, and a proper citation provided. Any violation of the University's policies on Academic and Professional Integrity may result in serious penalties, which might range from failing an assignment, to failing a course, to being expelled from the program. Violations of academic and professional integrity will be reported to Student Affairs. Consequences impacting assignment or course grades are determined by the faculty instructor; additional sanctions may be imposed.

Annotating in your group in a non-independent way is considered grounds for violation of Academic Integrity and will receive a zero for that part of the assignment. Code generated through automated or AI-assisted tools, such as Co-Pilot, will also receive a zero.

## References

- [1] Hyung Won Chung, Le Hou, Shayne Longpre, et al.  
*Scaling instruction-finetuned language models*. arXiv preprint arXiv:2210.11416, 2022.

- [2] Rowan Zellers, Ari Holtzman, Yonatan Bisk, et al.  
*HellaSwag: Can a machine really finish your sentence?* In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019.
- [3] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa.  
*Large language models are zero-shot reasoners.* Advances in Neural Information Processing Systems, 35:22199–22213, 2022.